Facens

Disciplina

Algoritmos e Programação

Conteúdo

Introdução aos conceitos de linguagem de programação estruturada em C

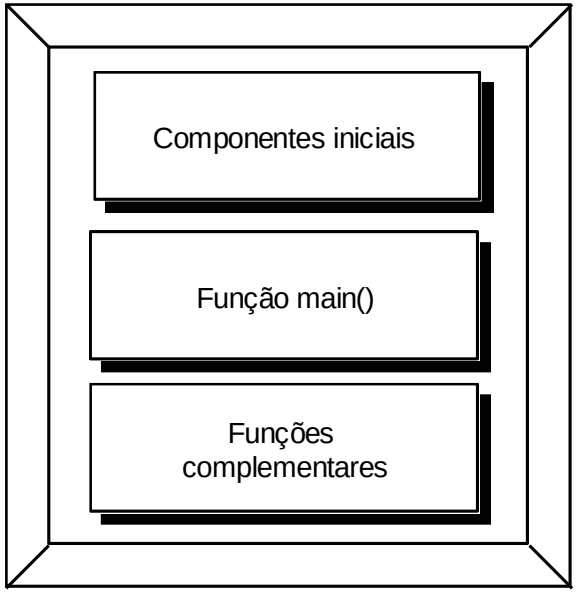
Ao final desta aula o estudante deverá

Familiarizar-se com Estruturas Lineares, Variáveis, Funções de Entrada/Saída e Operadores Aritméticos.

Facens

Estrutura de um programa em C

- Um programa fonte escrito em linguagem C pode ser composto de um ou mais arquivos, que devem ter o nome sempre com a extensão **.c**, sendo que sempre existe um arquivo principal do programa.
- No arquivo principal temos uma função especial chamada ***main*** **(principal)** que é por onde o programa irá iniciar a sua execução. Não pode haver duas funções *main* em um mesmo programa em linguagem C.
- Os componentes iniciais e as funções complementares podem existir no arquivo principal assim como nos arquivos complementares do programa em linguagem C.




```
#include <stdio.h>
float soma(float a, float b);

main( )
{
        float x, y, z;
        x=y=z=0;
        printf("\n Este programa calcula a expressão Z=X+Y ");
        printf("\n\n Digite o valor para x: ");
        scanf("%f", &x);
        printf("\n\n Digite o valor para y: ");
        scanf("%f", &y);
        z=soma(x, y);
        printf("\n A expressão z=x+y para x=%.2f e y=%.2f é %.2f ", x, y, z);
        return 0;
}

float soma(float a, float b)
{
        return (a + b);
}
```

Estrutura de um programa em C

- A linguagem C é uma linguagem estruturada, ou seja, ela é formada por blocos que são chamados de **funções**.
- A função main() tem a seguinte estrutura base:

```
main()
{
    //instruções
    return 0;
}
```

Função principal

Corpo do programa

"Retorno para o Sistema Operacional", opcional

Facens

Estrutura de um programa em C

- Além da **main( )** outras funções podem ser utilizadas no desenvolvimento dos programas, isto é, **dentro da main**().

Em C temos:

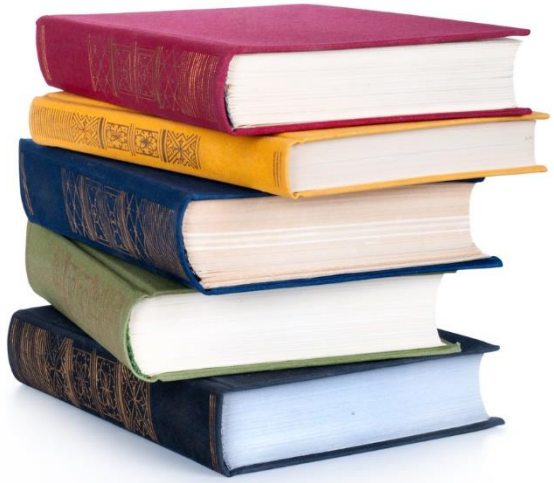
- **funções da biblioteca**
- funções definidas pelo usuário

- As principais funções podem ser adicionadas à main() deste que **sejam indicadas as bibliotecas** onde se encontram essas funções.

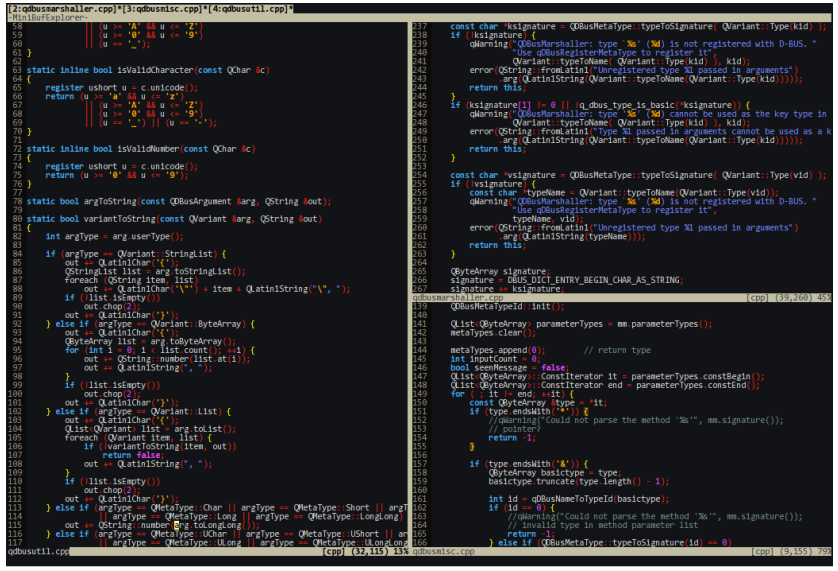
Facens

Bibliotecas

LIVROS



SCRIPTS



Facens

Bibliotecas

- **Bibliotecas** contêm funções pré-definidas, utilizadas nos programas.
  - A diretiva `#include` permite incluir uma biblioteca. Ela é seguida do nome da biblioteca entre sinais de maior (<) e menor (>).
  - Exemplos:

Diretiva + Biblioteca	Descrição
#include <stdio.h>	Funções de entrada e saída
#include <stdlib.h>	Funções tipo padrão
#include <math.h>	Funções matemáticas
#include <conio.h>	Funções para manipular caracteres na tela
#include <string.h>	Funções de texto

Facens

Estrutura de um programa em C

```
#include <stdio.h>
#include <stdlib.h>

main()
{
   printf("Exemplo de um Programa em C");
   return 0;
}
/* Este programa exibe uma mensagem na tela  */
ou
// Este programa exibe uma mensagem na tela
```

Cabeçalho (diretivas do pre-processador)

Bloco de instruções

Comentário

Facens

Estrutura de um programa em C

CUIDADOS:

- Finalizar as linhas com ponto-e-vírgula (;)
- A linguagem C é *case sensitive*, ou seja, letras maiúsculas são diferenciadas de letras **minúsculas**
- Sempre que tivermos um nome dentro do programa seguido de abertura e fechamento de parênteses, trata-se de uma **função**. Exemplo de funções: main(), printf() e getch()
- Os comentários de uma só linha são marcados pelo //, já os de múltiplas linhas são iniciados pelo /* e finalizados por */

Facens

Estrutura de um programa em C

```
#include <stdio.h>
#include <stdlib.h>

main()
{
   printf("Exemplo de um Programa em C");
   system("pause");
   return 0;
}
```

Só funciona no Windows!

Cabeçalho (diretivas do pre-processador)

printf( ) – stdio.h

system( ) – stdlib.h

Facens

Estrutura de um programa em C

```
#include <stdio.h>
#include <conio.h>

main()
{
   printf("Exemplo de um Programa em C");
   getch( );
   return 0;
}
```

Cabeçalho (diretivas do pre-processador)

printf( ) – stdio.h

getch( ) – conio.h

Facens

Variáveis

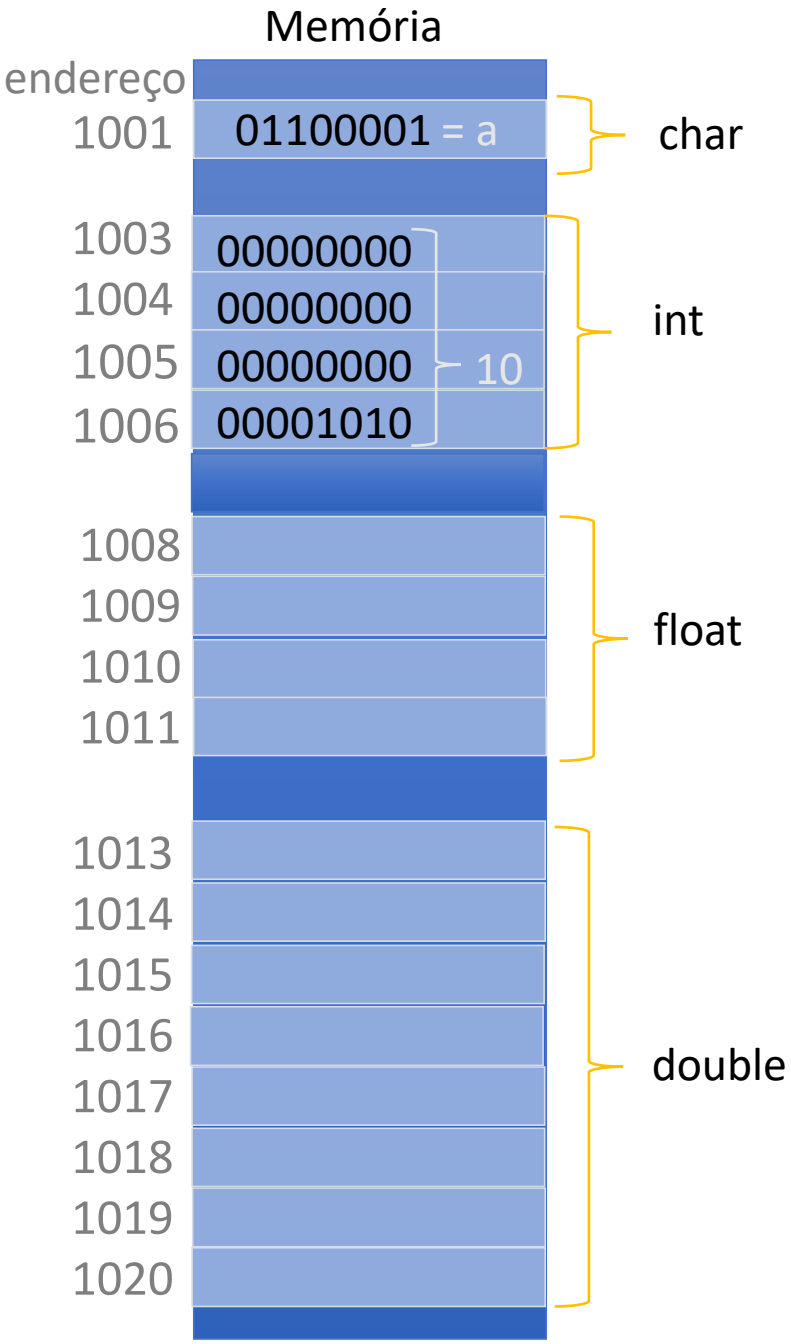
- **Variáveis** são espaços de memória reservados que guardam valores durante a execução de um programa.
- Todas as variáveis em C **devem** ser declaradas, antes de serem usadas.

Facens

Tipos de dados da linguagem C

- O **tipo** de uma variável informa a quantidade de memória, em bytes, que a variável ocupará e a forma como um valor deverá ser armazenado.
- Há cinco tipos básicos de dados em C.
  - char - apenas 1 caractere alfanumérico.
  - int - números inteiros.
  - float - números fracionários com precisão simples.
  - double - números fracionários com precisão dupla.
  - void - indica que é uma função e não tem retorno.

Facens



Tipos de dados da linguagem C

BIT – 0 ou 1 – é a menor partícula de informação em um computador, mas um único bit não consegue representar todas as letras, números e caracteres com os quais o computador trabalha. É necessário agrupá-los e cada grupo é chamado de **Byte**.

BYTE – é um grupo (conjunto) de 8 bits

Facens

Tipos de dados da linguagem C

Principais tipos de dados:

Tipo de dados Linguagem C	Tipo de dados Algoritmo	Tamanho aproximado	Faixa de Dados	Exemplos
int	inteiro	4 bytes	-2.147.483.648 a 2.147.483.647	-503 0 912
char	caractere	1 byte	-128 a 127	‘a’ ‘4’ ‘$’
float	real	4 bytes	Seis dígitos de precisão	-5.234432 0.912345 9832.368
double	real	8 bytes	Dez dígitos de precisão	-1.123456789 0.00001234 29396.232425
void	-	0 bytes	Nenhum valor	

Facens

Variáveis

Nomes de variáveis

- O nome de uma variável pode ser de uma letra até uma palavra com no máximo 32 caracteres.
- Pode conter letras, números e caractere de sublinhado (underline, underscore).
- O primeiro caractere do nome **não** pode ser numérico.
- Por convenção, variáveis são criadas com letras **minúsculas** (*case sensitive*).
  - **int variavel**;
  - int Variavel;
  - int VaRiAVeL;
  - int VARIAVEL;

    ou
  - int variavel, Variavel, VaRiAVeL, VARIAVEL;

Facens

Variáveis

Nomes de variáveis

- Não é permitido utilizar como nome de variáveis:
  - Nomes de funções criadas pelo programador ou pelas bibliotecas em C.
  - Palavras reservadas:

auto	*double*	*int*	*struct*
break	*else*	*long*	*switch*
case	*enum*	*register*	*typedef*
char	*extern*	*return*	*union*
const	*float*	*short*	*unsigned*
continue	*for*	*signed*	*void*
default	*goto*	*sizeof*	*volatile*
do	*if*	*static*	*while*

Facens

Variáveis

Declaração de variáveis

Sintaxe: **tipo nome_var;**

Exemplos:

```
char   letra;
int    x1, x2;
float  y, z, salario_minimo;
```

Inicialização de variáveis

Sintaxe: **nome_var = valor;**

Exemplos:

```
x1 = 10;
y = z = 10.5;
```

Declaração e inicialização de variáveis

Sintaxe: **tipo nome_var = valor;**

Exemplo:

```
int    x1 = 10;
```

Facens

Constantes

Constantes representam localizações de memória onde são armazenados dados que **não** podem ser modificados durante a execução do programa.

Nome de constantes

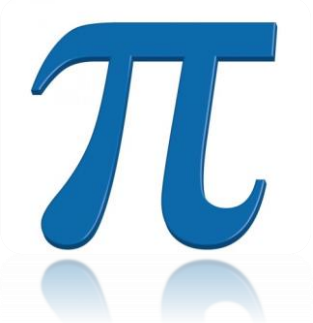
Por convenção, constantes são criadas com letras **maiúsculas**

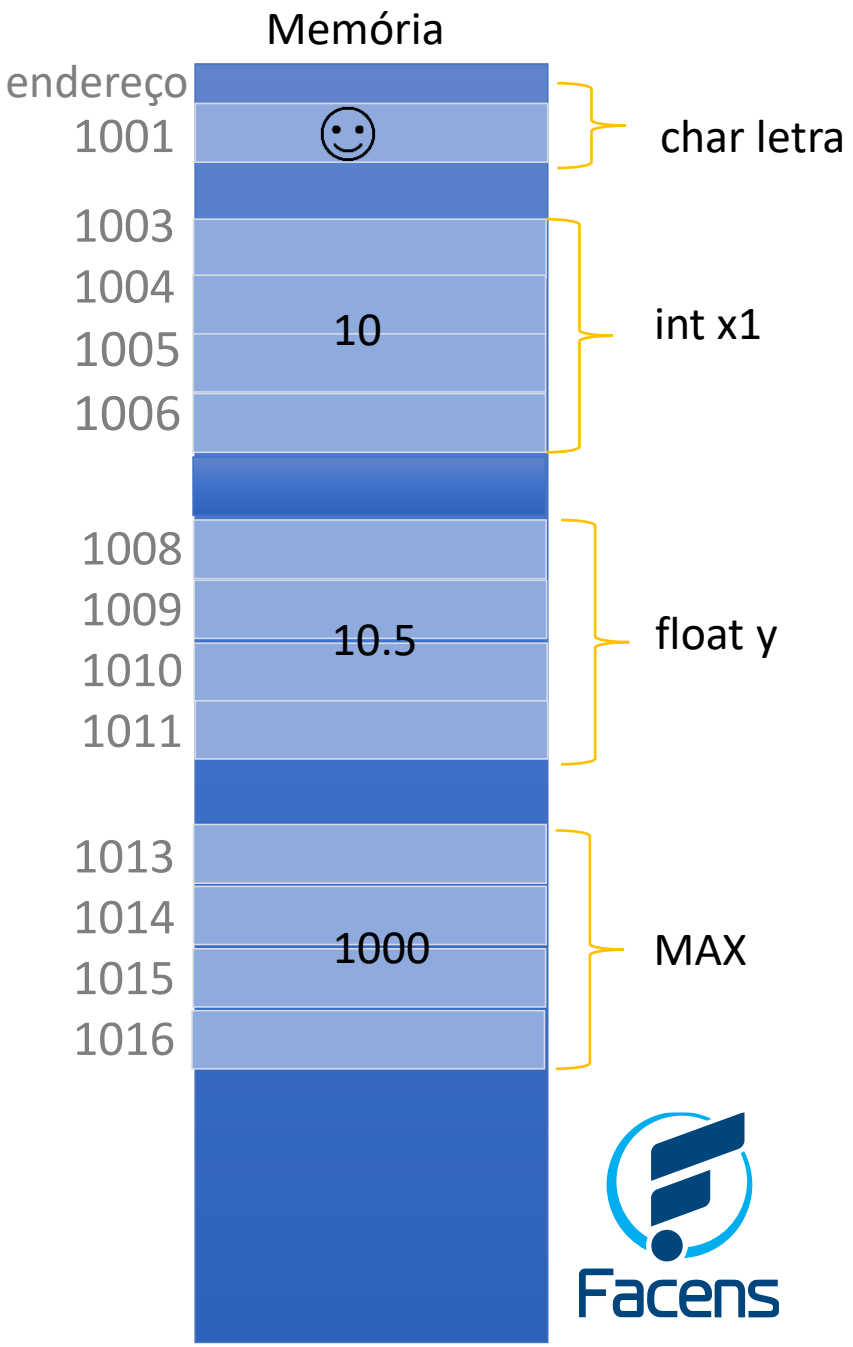
Declaração e inicialização de constantes

Sintaxe: **#define NOME_CONST valor**

Exemplos: #define PI 3.1415

#define MAX 1000





Resumo

Estruturas de um Programa

- **necessariamente** uma **única** função principal que chama-se **main()**
- Funções pré-definidas desde que indicada a **Biblioteca #include< >**
- Finalizar as linhas com ;
- Sempre **minúscula**

Variáveis e Constantes

- espaços de memória reservados que guardam **valores**
- Variáveis = pode **mudar** o valor, **minúscula**
- tipos básicos: **char, int, float, double, void**
- Constante = **não** pode mudar o valor, **maiúscula**

Facens

Variáveis

Exercício 1:

Verifique se o nome das variáveis abaixo são válidos:

a) total ✓
b) 1total ✗
c) char ✗
d) _vaLOR ✓
e) Valor1 ✓
f) ivalor ✓
g) float ✗

Facens

Variáveis

Exercício 2:

Declare e inicialize as seguintes variáveis/constantes:

a) numero = 4,56
b) valor = 13
c) PI = 3,1415
d) letra = C

Facens

Variáveis

Exercício 2:

Declare e inicialize as seguintes variáveis/constantes:

a) numero = 4,56

```
float numero;
numero = 4.56;
```

ou

```
float numero=4.56;
```

Facens

Variáveis

Exercício 2:

Declare e inicialize as seguintes variáveis/constantes:

b) valor = 13

```
int valor;
valor=13;
```

ou

```
int valor=13;
```

Facens

Variáveis

Exercício 2:

Declare e inicialize as seguintes variáveis/constantes:

c) PI = 3,1415

```
#define  PI   3.1415
```

Facens

Variáveis

Exercício 2:

Declare e inicialize as seguintes variáveis/constantes:

d) letra = C

```
char letra;
letra='C';
```

ou

```
char letra='C';
```

Facens

Códigos de formatação

Também chamados de especificadores de formato são utilizados para especificar o tipo de dado que será **lido** ou **escrito** na tela.

Código	Tipo de dados
%c	char (caracteres individuais)
%s	char (conjunto de caracteres)
%i ou %d	int
%f	float
%lf	double
%e	Número em notação científica, por exemplo, 10.3e-8
%x	Número em hexadecimal, por exemplo, 32F2
%p	Endereço de memória (ponteiro)
%%	Imprime um %

Facens

Função printf( )

- Biblioteca de funções **stdio.h**;
- A função **printf()** é utilizada para exibir os dados na tela.
- Sintaxe: **printf(a1, a2, a3, ..., an);**
  - Primeiro argumento (a1) chama-se <u>string de formato</u>, deve estar entre aspas duplas e pode conter:
    - Texto: **printf("Boa noite!");**
    - Códigos de barra invertida: **printf("Boa \n noite!");**
    - Especificadores de formato: **printf("O valor de x é %i",x);**

  - Demais argumentos (a2, ..., an) chamam-se <u>itens de dados</u>. São quantos forem os especificadores de formato do primeiro argumento
    - **printf("FLOAT = %f INT = %i CHAR = %c", var1, var2, var3);**
    - **printf("Resultado = %.2f ", num);**

Facens

Códigos de barra invertida

- Muitas vezes é necessário inserir código de barra invertida para formatar a saída do texto.

Código	Formato
\n	Nova linha.
\t	Tabulação.
\a	Beep – Toque do auto-falante.
\\	Barra invertida.
\0	Zero.
\’	Aspas simples (apóstrofo).
\”	Aspas duplas.
\xdd	Representação hexadecimal.
\ddd	Representação octal.

Facens

Função scanf( )

- Biblioteca de funções **stdio.h**.
- A função **scanf()** é utilizada para ler dados do teclado e colocar os valores fornecidos pelo usuário nas variáveis utilizadas como parâmetros da função.

Facens

Função scanf( )

- Sintaxe: **scanf(a1, a2, a3, ..., an);**
  - Primeiro argumento (a1) chama-se string de formato, deve estar entre aspas duplas e pode conter:
    - Especificadores de formato: **scanf("%i", &x);**
  - Demais argumentos (a2,...,an) chamam-se itens de dados. São quantos forem os especificadores de formato do primeiro argumento e devem ser precedidos pelo operador **& (endereço de)**
    - **scanf("%f %i %c", &var1, &var2, &var3);**

Facens

Operadores

A linguagem C é muito rica em operadores internos.

As classes de operadores são:

- Aritméticos;
- Relacionais;
- Lógicos.

Facens

Operador de Atribuição

- O símbolo de igual (=) representa a **atribuição** de um valor à uma variável, onde a variável e o valor atribuído devem ser de tipos compatíveis.
- Sintaxe: nome_var = expressão;
- Exemplos: x = 2;

  y = x + 3;
- **Conversão de tipos em atribuições**
  - Regra: o valor do lado direito de uma atribuição é convertido para o tipo do lado esquerdo
  - Exemplo: int x;

    float y;

    x = y = 3.5;

Facens

Operadores Aritméticos

Binários – atuam sobre dois operandos:

Principais Operadores Aritméticos da Linguagem C		
Operação	**Operador**	**Exemplo**
Adição	+	8 ← 5 + 3
Subtração	-	7 ← 9 - 2
Multiplicação	*	8 ← 2 * 4
Divisão	/	2 ← 6 / 3
Módulo (resto da divisão inteira)	%	2 ← 5 % 3

Facens

Operadores Aritméticos

Binários – Esses operadores possuem uma ordem de precedência.

Ordem dos Operadores	
Ordem	**Operador**
1º	Parênteses mais internos
2º	Potenciação e raiz
3º	* e /
4º	+ e -

Facens

Operadores Aritméticos

A linguagem C também oferece funções predefinidas:

- Para inteiros (stdlib.h):
  - abs(x): retorna o valor absoluto de x.
- Para double (math.h):
  - atan(x): retorna arco tangente de x.
  - cos(x): retorna cosseno de x.
  - sin(x): retorna seno de x.
  - exp(x): retorna função exponencial natural (e^x).
  - pow(x,y): eleva x à potência y.
  - sqrt(x): retorna a raiz quadrada de x.
  - log(x): retorna o logaritmo natural de x.

Facens

Pausar o programa

- Existem duas possibilidades de solicitar uma pausa na execução do programa.
- A pausa irá permitir a verificação da saída de um programa.
- É possível utilizar dois comandos para essa finalidade:
  - `system("PAUSE");` //Biblioteca #include<stdlib.h> (só funciona no Windows!)
  - `getch();` //Biblioteca #include<conio.h>

Facens

Variáveis e Operadores Aritméticos

Exercício 3:

- Escreva o que será mostrado no monitor em cada uma das sentenças abaixo:
  a) printf("Aula de \"Algoritmos\" \\o/\n");
  b) printf("um\n\tdois\n\t\ttres\n");
  c) printf("Produto %c - %.2f%% de desconto\n",'A',7.5);
  d) printf("%i\n", 2/3);
  e) printf("%.1f\n", 2.0/3.0);
  f) printf("%i / %.2f = %f\n", 3, 2.0, 3.0/2);

Facens

Organização de um programa

- Como organizar um programa em C:

```
//Bibliotecas

//Constantes
int main() //Função principal
{
    //Declaração de variáveis

    //Entrada

    //Processamento

    //Saída
    //Pausa o programa - se necessário
    return 0; //Finaliza o programa
}
```

Facens

Operadores

Exercício 4:

Receber dois valores, calcular a sua soma e exibir o resultado.

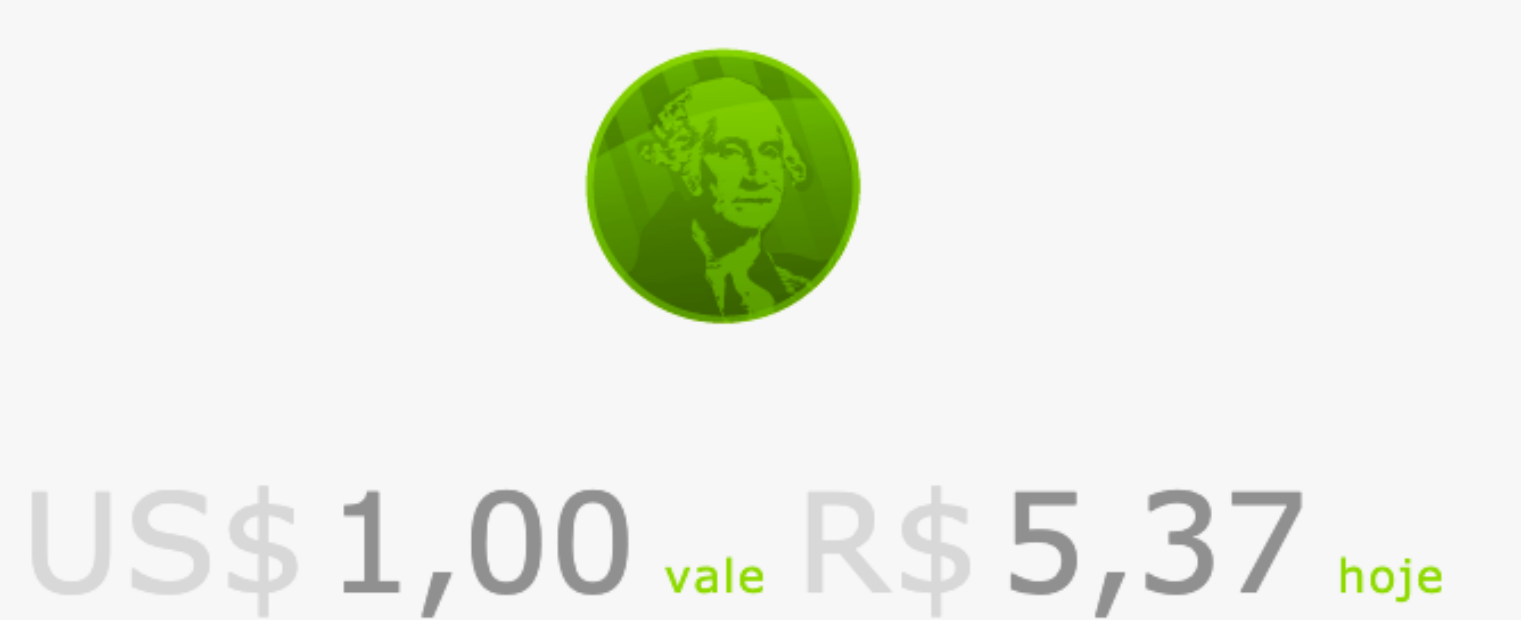
```
#include<stdio.h>
#include<conio.h>
int main()
{
    int n1,n2,soma;
    printf("Digite o primeiro numero:");
    scanf("%i",&n1);
    printf("Digite o segundo numero:");
    scanf("%i",&n2);
    soma = n1+n2;
    printf("\nResultado = %i",soma);
    getch();
    return 0;
}
```

Facens

Operadores

Exercício 5:

Dado o preço do produto em reais, converta esse valor para o equivalente em dólares. O programa deverá ler o valor em reais e a taxa de conversão para dólar.



Facens

Operadores

Exercício 5:

Dado o preço do produto em reais, converta esse valor para o equivalente em dólares. O programa deverá ler o valor em reais e a taxa de conversão para dólar.

```
#include<stdio.h>
#include<conio.h>
int main()
{
    float real, cotacao, dolar;
    printf("Digite o valor do produto e o valor do dolar hoje: ");
    scanf("%f %f",&real,&cotacao);
    dolar = real/cotacao;
    printf("R$ %.2f => US$ %.2f.",real,dolar);
    getch();
    return 0;
}
```

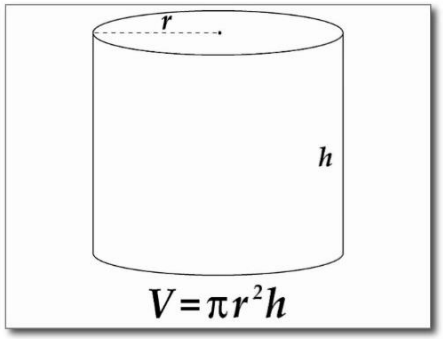
Facens

Operadores

Exercício 6:

Implemente um programa em C que calcule o volume (V) de um cilindro através dos valores da altura (h) do cilindro e do raio (r) da base.

O volume é calculado através da seguinte fórmula:



Considere o valor de PI=3,14159.

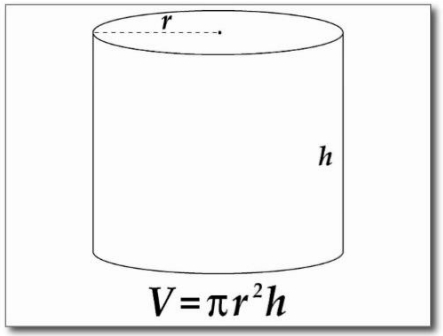
Facens

Operadores

Exercício 6:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>

#define PI 3.14159
int main()
{
    float altura, raio, volume;
    printf("Digite a altura e o raio:");
    scanf("%f %f",&altura,&raio);
    volume = PI * pow(raio,2) * altura;
    printf("Volume = %f.",volume);
    getch();
    return 0;
}
```



Facens

Operadores Aritméticos

Unários – atuam sobre apenas um operando

Operadores Aritméticos Unários da Linguagem C		
Operação	**Operador**	**Exemplo**
Menos Unário	-	-x → x * (-1) // Multiplica a variável por -1
Incremento	++	++x → x = x + 1 // Incrementa variável antes de usá-la x++ → x = x + 1 // Incrementa variável depois de usá-la
Decremento	--	--x → x = x – 1 // Decrementa variável antes de usá-la x-- → x = x – 1 // Decrementa variável depois de usá-la

Facens

Operadores Aritméticos

Exercício 7:

Qual o valor das variáveis x, y e z depois da seguinte sequência de operações?

```
int x, y, z;
x = y = 10;
z = (x++);
x = -x;
y++;
x = x + y – (z--);
```

x	y	z
10	10	-
11	10	10
-11	10	10
-11	11	10
-10	11	9

Facens

Operadores Relacionais

São usados para **comparar expressões** e resultam em falso ou verdadeiro.

Operadores de Igualdade			
Operador	**Símbolo**	**Exemplo**	**Resultado**
Igualdade	==	10 == 20	Falso
Diferença	!=	10 != 20	Verdadeiro

Operadores Relacionais			
Operador	**Símbolo**	**Exemplo**	**Resultado**
Maior que	>	10 > 20	Falso
Menor que	<	10 < 20	Verdadeiro
Maior ou igual que	>=	5 >= 5	Verdadeiro
Menor ou igual que	<=	12 <= 20	Verdadeiro

Facens

Operadores Lógicos

São usados para relacionar duas expressões ou apenas uma (que é o caso do operador de negação).

- **Operador AND (&&)**
  - Resulta em verdadeiro se ambas expressões forem verdadeiras
- **Operador OR (||)**
  - Resulta em verdadeiro se pelo menos uma expressão for verdadeira
- **Operador NOT (!)**
  - Resulta em verdadeiro se a expressão for falsa

Facens

Operadores Lógicos

Operadores lógicos utilizados entre A e B que no exemplo são expressões com operadores relacionais.

Operadores Lógicos			
Operador	**Representação na Lógica Matemática**	**Representação na Linguagem C**	**Exemplo**
Conjunção - AND	E (∧)	&&	A && B
Disjunção - OR	OU (∨)	||	A || B
Negação - NOT	Negação (~)	!	!A

Facens

Operadores Lógicos

Exercício 8:

Verifique o retorno do valor de cada expressão abaixo:

• 1 == 1 && 2 == 2	• retorna **verdadeiro**
• 2 > 1 && 2 != 2	• retorna **falso**
• 1 != 1 || 2 != 2	• retorna **falso**
• 2 >= 1 || 2 != 2	• retorna **verdadeiro**
• !(2 > 2)	• retorna **verdadeiro**
• !(2 == 2)	• retorna **falso**

Facens

Exercícios

Caderno de Exercício de Algoritmos e Programação

Lista 1 – Comando de Entrada e Saída, Operações e Variáveis

Facens